UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ingeniería en Tecnologías de la Información e Innovación Digital

ACTIVIDAD 4

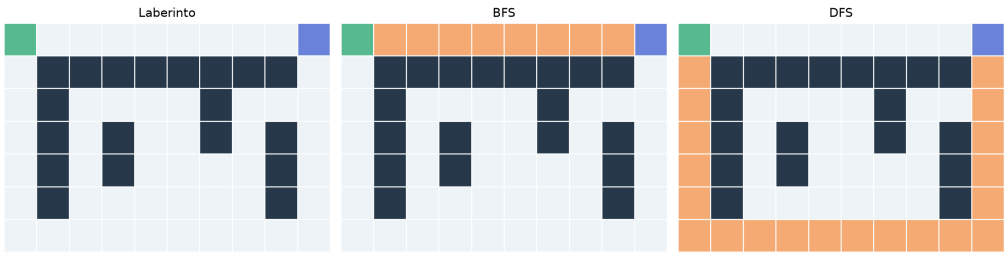
Búsqueda ciega: BFS y DFS

Datos de entrega	Información
Asignatura	Fundamentos de Inteligencia Artificial
Alumno	Emmanuel Guzmán
Grupo	172
Docente	Mtro. Javier García Lara
Fecha indicada en el enunciado	25 de octubre de 2026

Portafolio individual versionado en GitHub. Se incluyen planteamiento, procedimiento, evidencia, conclusiones y referencias.

Planteamiento y representación

Resolver un laberinto didáctico de 7 × 10 celdas. `#` es pared, `S` inicio y `G` meta. Cada celda libre es un estado (fila, columna); las acciones legales mueven arriba, abajo, izquierda o derecha a una celda libre, con costo 1. El grafo es no dirigido y no ponderado. Se construyó con 51 estados y 63 aristas.



Procedimiento

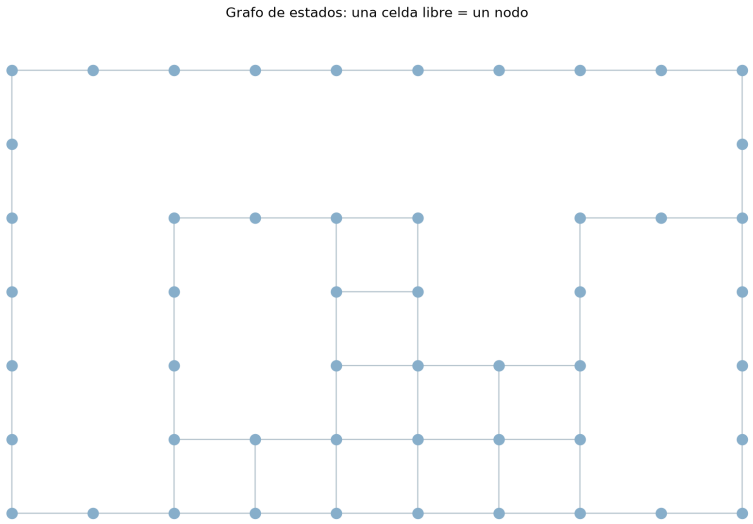
BFS usa una cola FIFO y descubre estados por distancia creciente; DFS usa una pila LIFO y profundiza en una rama. Ambos registran padres al descubrir un estado para reconstruir la ruta y evitan ciclos con visitados. El orden de vecinos se fija por (fila, columna); DFS extrae primero el vecino mayor de la pila. Cada algoritmo termina al extraer la meta.

Resultados de la ejecución

Métrica	BFS	DFS
Longitud de ruta	9 pasos	21 pasos
Nodos expandidos	18	37
Máximo en frontera	3	11
Tiempo (una ejecución)	36.2 µs	47.1 µs

El tiempo es una sola medición con reloj de alta resolución y cambia entre equipos y ejecuciones; la longitud de ruta y nodos expandidos son las medidas más útiles para comparar este ejemplo. BFS garantiza ruta mínima con costo uniforme; DFS depende del orden y aquí encuentra una más larga.

Grafo de estados



Complejidad y código

En el grafo finito explícito, ambos recorren como máximo V estados y E aristas: O(V+E) tiempo y O(V) memoria por padres y frontera. La versión clásica en árbol se expresa como BFS O(b^d) tiempo/memoria y DFS O(b^m) tiempo, O(bm) memoria, con factor de ramificación b, profundidad óptima d y profundidad máxima m. El código está en **Act4/act4_laberinto.py**; las pruebas verifican adyacencia de cada paso, inicio, meta y que BFS no es más largo que DFS.

Conclusión

BFS conviene si importa la ruta más corta y la memoria disponible alcanza; DFS puede ser útil para explorar con una pila, pero no garantiza la solución mínima. La selección del algoritmo debe responder a la meta real, al tamaño del espacio y al costo de las acciones.

Referencias (APA 7.ª ed.)

Russell, S., & Norvig, P. (2020). Artificial intelligence: A modern approach (4th ed.). Pearson.

Bohm, R. A. (2021). The fundamentals of search algorithms. Nova Science Publishers.

Documentación oficial de NetworkX. (s. f.). Algorithms. https://networkx.org/documentation/stable/reference/algorithms/

Uso de inteligencia artificial y verificación

Se utilizó ChatGPT/Codex para apoyar la organización del reporte, la redacción, el desarrollo de código y gráficos. Se ejecutaron los programas y pruebas locales, y se cotejaron las afirmaciones de Act1 con las fuentes citadas. El estudiante debe revisar, comprender y poder explicar esta entrega antes de presentarla.

Anexo: código Python completo

Archivo ejecutable **Act4/act4_laberinto.py**. La numeración facilita la revisión; el archivo del repositorio es la versión directamente ejecutable.

```
001  """BFS y DFS iterativos sobre un laberinto de celdas libres."""
002  from __future__ import annotations
003
004  import json
005  import time
006  from collections import deque
007  from pathlib import Path
008
009  import matplotlib
010  matplotlib.use("Agg")
011  import matplotlib.pyplot as plt
012  import networkx as nx
013
014  BASE = Path(__file__).resolve().parent
015  OUT = BASE / "resultados"
016  MAPA = [
017      "S........G",
018      ".########.",
019      ".#....#...",
020      ".#.#..#.#.",
021      ".#.#....#.",
022      ".#......#.",
023      "..........",
024  ]
025
026
027  def construir_grafo(mapa: list[str] = MAPA) -> tuple[nx.Graph, tuple[int,int], tuple[int,int]]:
028      if not mapa or len({len(fila) for fila in mapa}) != 1:
029          raise ValueError("Mapa vacio o no rectangular")
030      grafo = nx.Graph()
031      inicio = meta = None
032      for y, fila in enumerate(mapa):
033          for x, celda in enumerate(fila):
034              if celda == "#":
035                  continue
036              grafo.add_node((y, x))
037              if celda == "S": inicio = (y, x)
038              if celda == "G": meta = (y, x)
039              for previo in ((y-1, x), (y, x-1)):
040                  if previo in grafo:
041                      grafo.add_edge((y, x), previo)
042      if inicio is None or meta is None:
043          raise ValueError("Falta S o G")
044      return grafo, inicio, meta
045
046
047  def buscar(grafo: nx.Graph, inicio: tuple[int,int], meta: tuple[int,int],
048             metodo: str) -> tuple[list[tuple[int,int]], int, int]:
049      """Devuelve ruta, nodos expandidos y máximo de frontera."""
050      if metodo not in {"BFS", "DFS"}:
051          raise ValueError("Metodo debe ser BFS o DFS")
052      frontera = deque([inicio])
053      padres = {inicio: None}
054      expandidos = 0
055      max_frontera = 1
056      while frontera:
057          actual = frontera.popleft() if metodo == "BFS" else frontera.pop()
058          expandidos += 1
059          if actual == meta:
060              ruta = []
061              while actual is not None:
062                  ruta.append(actual)
063                  actual = padres[actual]
064              return list(reversed(ruta)), expandidos, max_frontera
065          vecinos = sorted(grafo.neighbors(actual))
066          # En DFS la pila visita primero el vecino mayor en orden (fila, columna).
067          for vecino in vecinos:
068              if vecino not in padres:
069                  padres[vecino] = actual
070                  frontera.append(vecino)
071          max_frontera = max(max_frontera, len(frontera))
072      return [], expandidos, max_frontera
073
074
075  def dibujar_mapa(rutas: dict[str,list[tuple[int,int]]]) -> None:
076      fig, ejes = plt.subplots(1, 3, figsize=(14, 4))
077      for ax, (nombre, ruta) in zip(ejes, [("Laberinto", []), *rutas.items()]):
078          for y, fila in enumerate(MAPA):
079              for x, celda in enumerate(fila):
080                  color = "#26384a" if celda == "#" else "#edf3f7"
```

```
                if (y, x) in ruta: color = "#f4aa72"
                if celda == "S": color = "#56b98f"
                if celda == "G": color = "#6a82d9"
                ax.add_patch(plt.Rectangle((x, y), 1, 1, facecolor=color, edgecolor="white"))
        ax.set_xlim(0, len(MAPA[0])); ax.set_ylim(len(MAPA), 0)
        ax.set_aspect("equal"); ax.axis("off"); ax.set_title(nombre)
    fig.tight_layout()
    fig.savefig(OUT / "act4_rutas.png", dpi=180)
    plt.close(fig)


def ejecutar() -> dict:
    OUT.mkdir(exist_ok=True)
    grafo, inicio, meta = construir_grafo()
    datos = {"mapa": MAPA, "estados_libres": grafo.number_of_nodes(),
             "aristas": grafo.number_of_edges(), "inicio": inicio, "meta": meta}
    rutas = {}
    for metodo in ("BFS", "DFS"):
        t0 = time.perf_counter_ns()
        ruta, expandidos, max_frontera = buscar(grafo, inicio, meta, metodo)
        tiempo_us = (time.perf_counter_ns() - t0) / 1000
        rutas[metodo] = ruta
        datos[metodo] = {"ruta": ruta, "pasos": len(ruta)-1,
                         "nodos_expandidos": expandidos, "max_frontera": max_frontera,
                         "tiempo_microsegundos_una_ejecucion": tiempo_us}
    dibujar_mapa(rutas)
    pos = {(y,x):(x,-y) for y,x in grafo.nodes}
    fig, ax = plt.subplots(figsize=(11, 7))
    nx.draw(grafo, pos, ax=ax, node_size=80, node_color="#87aeca", edge_color="#b3c2cc")
    ax.set_title("Grafo de estados: una celda libre = un nodo")
    fig.tight_layout(); fig.savefig(OUT / "act4_grafo.png", dpi=170); plt.close(fig)
    (OUT / "act4_resultados.json").write_text(json.dumps(datos, indent=2), encoding="utf-8")
    print(json.dumps(datos, indent=2))
    return datos


if __name__ == "__main__":
    ejecutar()
```